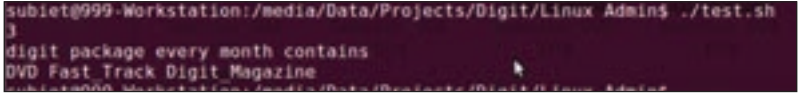


```
# Displaying No. Elements in array
echo ${#Digit_Contents[@]}
# Display individual elements
echo 'digit package every month contains'
```



```
subiet@999-Workstation:/media/Data/Projects/Digit/Linux Admin$ ./test.sh
3
digit package every month contains
DVD Fast Track Digit Magazine
```

Introduction to arrays

```
echo ${Digit_Contents[0]} ${Digit_Contents[1]} ${Digit_Contents[2]}
```

Just the variable name `$Digit_Contents` always refer to the first element is equivalent to writing `${Digit_Contents[0]}`.

Please note that the names are case sensitive, so `Digit` is not equal to `digit`.

Executing and quotes

As a first step towards executing your script you would need to explicitly grant it executable permissions. Some modern distributions after noticing a shebang symbol might automatically store it with executable permissions. Just to be sure, do the following in the directory where your script is stored.

```
$ chmod +x script_name.sh
```

To execute this script, issue the following at the prompt

```
$ ./script_name.sh
```

the `/` symbol is required to tell the shell that it needs to interpret/execute the script and not look for a program to open the file.

It is imperative to note that each command in the bash script can individually be executed sequentially on the shell also, because it is an interpreted language. So the question is can bash commands be also executed from within a script, and the answer is very loud yes because it is a very frequently used and useful feature. Executing shell commands with bash can be archived with backticks, to illustrate it, consider the following example

```
#!/bin/bash
```

```
# use backticks "`" to execute shell command
```

```
echo `ls -l`
```

```
# executing bash command without backticks
```

```
echo
```

```
echo
```

```
echo ls -l
```

In the above example the first `echo`, would perform the same way as issuing the `ls -l` command on the prompt. The last `echo` would do nothing